SecureCloud+: Secure Cloud File Storage using Dynamic AES-256-GCM, ECC-Based Key Protection, Blockchain Key Management, Fine-Grained Access Control, Key Revocation, and Key Rotation

Sejal R. Chinchkar
Department of Computer Engineering
SecureCloud+ Research Project

***Abstract*—Cloud storage outsourcing exposes sensitive digital assets to untrusted infrastructure, insider exfiltration, and unauthorized data tampering. Traditional cloud storage paradigms rely on static symmetric encryption keys, centralized key-management servers that represent single points of compromise, or coarse-grained access control models that lack dynamic post-revocation guarantees. This paper presents **SecureCloud+**, a hybrid cryptographic architecture for secure cloud file storage that seamlessly combines dynamic symmetric payload encryption, asymmetric elliptic curve key encapsulation, deterministic rule-based access control, automated cryptographic key rotation upon revocation, and decentralized metadata anchoring. The system employs AES-256 in Galois/Counter Mode (AES-256-GCM) with fresh per-file initialization vectors and dynamic keys to guarantee at-rest confidentiality and ciphertext authenticity. Symmetric file keys are encapsulated using Elliptic Curve Diffie-Hellman (ECDH) over the SECP256R1 (NIST P-256) curve combined with HKDF-SHA256, isolating key distribution to designated public-key recipients without exposing plaintext keys to cloud storage operators. Access control is enforced at the API route-guard layer across four granular permission states (`OWNER`, `READ`, `DOWNLOAD`, `DENIED`), preemptively halting unauthorized requests prior to cryptographic processing. When access is revoked by a file owner, the architecture triggers automated key rotation: recovering the active file key, generating a fresh 256-bit AES key, re-encrypting the stored payload, incrementing the file version number ($v1 \rightarrow v2$), re-wrapping the key for remaining authorized grantees, and logging an immutable event on an Ethereum Virtual Machine (EVM) smart contract. Empirical evaluation across five discrete payload sizes (10 KB to 5 MB) demonstrates that AES-256-GCM encryption and decryption scale within 0.0698–5.6952 ms and 0.0156–4.6281 ms, respectively. Complete upload and download pipelines execute within 142.79–184.57 ms and 33.51–48.56 ms, while key rotation with payload re-encryption executes in 108.76–140.52 ms. Blockchain transaction times remain stable across payload sizes (96.75–108.08 ms) by recording fixed-size metadata alone. An automated test suite of 50 unit and integration tests achieved 100% pass rates (0 failures), and eight formal security validation experiments confirmed zero on-chain key leakage, immediate detection of ciphertext bit tampering, robust rejection of mismatched private keys, and pre-decryption rejection of revoked users with `HTTP 403 Forbidden`.

***Keywords*—Cloud data security, AES-256-GCM, Elliptic Curve Cryptography, SECP256R1, ECDH key exchange, HKDF-SHA256, dynamic key rotation, rule-based access control, blockchain auditability, smart contracts.**

I. INTRODUCTION

Outsourcing data storage to public and commercial cloud infrastructures offers undeniable operational advantages, including elastic storage allocation, multi-tenant accessibility, and reduced capital expenditures for enterprise IT infrastructure. However, placing organizational and personal data on third-party cloud servers fundamentally shifts the custody of sensitive assets outside the data owner's physical and administrative security perimeter. In a conventional cloud deployment, storage hosts operate under an *honest-but-curious* (semi-trusted) operational model: while service providers faithfully execute storage and retrieval commands, they cannot be inherently trusted with data confidentiality or unmonitored access integrity. Storage volumes remain susceptible to passive inspection, privileged insider threats, rogue system administrators, hypervisor-level breaches, and persistent external network attacks.

To mitigate unauthorized data exposure, client-side symmetric encryption has become standard practice in secure cloud storage architectures. By encrypting data prior to transmission, organizations ensure that storage servers encounter only unreadable ciphertext blocks. Among symmetric algorithms, the Advanced Encryption Standard (AES) operating with 256-bit keys is widely recognized as the industry standard. However, conventional block cipher modes such as Electronic Codebook (ECB) and Cipher Block Chaining (CBC) fail to provide comprehensive data authenticity. ECB preserves data patterns across identical blocks, while CBC requires separate, non-integrated Message Authentication Codes (MACs) to defend against bit-flipping attacks, padding-oracle vulnerabilities, and chosen-ciphertext manipulation.

Authenticated Encryption with Associated Data (AEAD), specifically AES-256 in Galois/Counter Mode (AES-256-GCM) [2], resolves these structural weaknesses by pairing high-performance counter-mode stream encryption with universal-hash-based Galois field multiplication (GHASH). This dual-function architecture simultaneously guarantees payload confidentiality and cryptographic authenticity. Any unauthorized alteration of the stored ciphertext, initialization vector, or accompanying authentication metadata induces an immediate

GHASH tag mismatch, causing the decryption engine to reject the modified payload before releasing plaintext bytes.

While authenticated symmetric encryption secures data payloads, the management, encapsulation, and distribution of symmetric encryption keys remains the foremost structural challenge in secure multi-user cloud systems. Centralized Key Management Interoperability Protocol (KMIP) servers or Key Management Services (KMS) introduce severe architectural vulnerabilities: they function as single points of catastrophic failure, present high-value targets for credential compromise, and require clients to place implicit trust in external key custody providers. Furthermore, distributing static symmetric keys directly to multi-user workgroups makes selective revocation virtually impossible without compromising group-wide access.

Recently, Shakor et al. [1] proposed an innovative framework in *IEEE Access* entitled *"Dynamic AES Encryption and Blockchain Key Management: A Novel Solution for Cloud Data Security."* Their architecture demonstrated that dynamically generating fresh symmetric encryption keys for each file and anchoring key-management metadata into a decentralized blockchain ledger dramatically improves cloud storage security. By shifting key tracking from centralized databases to distributed smart contracts, their model eliminates single points of failure, ensures tamper-evident auditability, and provides decentralized key verification.

Despite these pioneering advances, a rigorous architectural and cryptographic analysis of the base model proposed by Shakor et al. reveals critical operational limitations when extended to multi-user collaborative cloud environments: (1) absence of user revocation mechanisms, (2) static key retention vulnerabilities, (3) lack of cryptographic payload re-encryption, (4) coarse access gating, and (5) absence of synchronous blockchain revocation event logging.

To address these fundamental research gaps, this paper presents **SecureCloud+**, a comprehensive, defense-in-depth cloud file storage system that extends the foundational concepts of dynamic symmetric encryption and blockchain key management into a fully integrated, multi-user access control and key lifecycle framework. SecureCloud+ is implemented and evaluated as a functioning software prototype integrating a high-performance Python/FastAPI backend, the native `cryptography` engine, an Ethereum Virtual Machine (EVM) Solidity smart contract (`FileKeyManager.sol`), and a Web3 integration layer.

The core scientific and engineering contributions of SecureCloud+ are structured across six key pillars: (1) dynamic authenticated payload encryption, (2) hybrid asymmetric key encapsulation via SECP256R1 and HKDF-SHA256, (3) deterministic pre-decryption route guarding, (4) automated key rotation and payload re-encryption, (5) decentralized, tamper-evident audit logging on an EVM smart contract, and (6) empirical benchmarking and formal security validation across 50 automated tests and eight formal security experiments.

II. RELATED WORK

A. Cloud Data Security and Authenticated Encryption

Early approaches to cloud storage security focused primarily on symmetric block encryption operating in CBC or CTR modes. Dworkin [2] formalized Galois/Counter Mode (AES-GCM), which combines counter-mode encryption with universal polynomial hashing over GF(2^{128}). Contemporary cloud storage systems overwhelmingly adopt AES-256-GCM due to its native hardware acceleration (AES-NI and CLMUL instruction sets) and robust authenticated encryption guarantees. SecureCloud+ builds directly upon NIST SP 800-38D [2] and FIPS PUB 180-4 [3] to enforce authenticated encryption and pre-encryption integrity digests.

B. Public-Key Cryptography and ECC-Based Key Management

Asymmetric key encapsulation provides the standard mechanism for distributing symmetric keys without preshared secrets. While RSA historically dominated key exchange, Elliptic Curve Cryptography (ECC) achieves equivalent cryptographic strength with substantially smaller key sizes. Barker et al. [5] standardized pair-wise key establishment using Elliptic Curve Diffie-Hellman (ECDH). To derive uniform cryptographic keys from elliptic curve points, Krawczyk and Eronen [4] specified the HMAC-based Extract-and-Expand Key Derivation Function (HKDF-SHA256). In SecureCloud+, SECP256R1 (NIST P-256) ECDH is paired with HKDF-SHA256 to establish Key Encryption Keys (KEKs) that wrap symmetric file keys for authorized grantees.

C. Blockchain in Cloud Storage and Key Management

The emergence of distributed ledger technology introduced tamper-resistant primitives for cloud metadata management. Lin et al. [6] proposed consortium blockchain frameworks for multi-cloud data integrity verification. Zang et al. [7] explored blockchain-anchored storage over cloud-native edge architectures. Liu et al. [8] developed zero-trust sharding mechanisms for cross-domain data sharing. Most significantly, Shakor et al. [1] demonstrated that recording dynamic AES key metadata on an EVM-compatible blockchain successfully decentralizes key verification and mitigates central KMS compromise. SecureCloud+ adopts this decentralized metadata anchoring paradigm but resolves the critical absence of revocation mechanisms in the base model.

D. Access Control and Revocation in Outsourced Storage

Fine-grained access control in cloud repositories has historically relied on Attribute-Based Encryption (ABE). Xiao et al. [9] proposed blockchain-based access control with flexible attribute revocation. Yang et al. [10] combined smart contracts with edge nodes to enforce fine-grained access policies for electronic health records. Luo et al. [11] presented key-policy ABE schemes with switchable attributes. However, ABE schemes introduce severe bilinear pairing computation overheads that scale poorly for large file repositories. Conversely, Yao et al. [12] and Susilo et al. [13] investigated proxy re-encryption (PRE) and sanitizable access control.

SecureCloud+ achieves deterministic, lightweight access control by coupling route-guard permission enforcement (`OWNER`, `READ`, `DOWNLOAD`, `DENIED`) with synchronous, owner-directed AES key rotation.

TABLE I: COMPARATIVE ANALYSIS OF RELATED CLOUD STORAGE AND ACCESS CONTROL ARCHITECTURES

Architecture / Study	Payload Cipher	Key Encapsulation	Blockchain Role	Pre-Decryption Guard	Key Rotation
Shakor et al. [1] (Base)	AES-256 (Dynamic)	None (Plain Meta)	Key Meta Registry	No	No
Lin et al. [6]	Static AES	RSA-2048	Integrity Auditing	No	No
Zang et al. [7]	AES-CBC	ECC Secp256k1	Block Hash Logging	Partial	No
Liu et al. [8]	AES-GCM	Identity-Based Enc	Zero-Trust Sharding	Yes	Manual
Xiao et al. [9]	CP-ABE	Bilinear Pairing	Policy Enforcement	Yes	Attribute Rev
Yang et al. [10]	Hybrid ABE	Attribute Tokens	Access Token Ledger	Yes	Proxy-Based
Yao et al. [12]	Proxy Re-Enc	Bilinear Pairing	None (Centralized)	No	Proxy-Re-Enc
Susilo et al. [13]	Sanitizable ABE	Pairing Groups	None (Centralized)	Yes	Periodic
SecureCloud+ (Proposed)	AES-256-GCM	SECP256R1 + HKDF	Metadata & Events	Yes (Deterministic)	Dynamic Auto

III. PROPOSED SYSTEM AND METHODOLOGY

A. Mathematical Cryptographic Formulation

Let P be the plaintext file payload. The system samples a dynamic 256-bit symmetric key K_{file} and a 96-bit initialization vector IV_{file}. AES-256-GCM produces ciphertext C_{file} and a 128-bit authentication tag T_{file}:

$$(C_{file}, T_{file}) = AES\text{-}256\text{-}GCM\text{-}Enc_{K_{file}}(IV_{file}, P, A) \quad (1)$$

Decryption reconstructs plaintext only if the GHASH tag validates:

$$P = AES\text{-}256\text{-}GCM\text{-}Dec_{K_{file}}(IV_{file}, C_{file}, T_{file}) \quad (2)$$

Elliptic curve key pairs operate on the SECP256R1 (NIST P-256) curve:

$$y^2 \equiv x^3 - 3x + b \pmod{p} \quad (3)$$

The shared secret point is derived via ECDH point multiplication:

$$S = d_{eph} \cdot Q_B = d_{eph} \cdot d_B \cdot G \quad (4)$$

The 256-bit Key Encryption Key (K_{KEK}) is extracted using HKDF-SHA256 [4]:

$$K_{KEK} = HKDF\text{-}SHA256(IKM = S_x, info = \text{"SecureCloud+ ECC-AES-GCM KEK"}, L = 32) \quad (5)$$

The dynamic file key is wrapped using AES-256-GCM:

$$(C_{key}, T_{key}) = AES\text{-}256\text{-}GCM\text{-}Enc_{K_{KEK}}(IV_{KEK}, K_{file}) \quad (6)$$

Recipient B decapsulates K_{file} using private scalar d_{B}:

$$K_{file} = AES\text{-}256\text{-}GCM\text{-}Dec_{K_{KEK}}(IV_{KEK}, C_{key}, T_{key}) \quad (7)$$

The pre-encryption payload is hashed via SHA-256 [3]:

$$H_{file} = SHA\text{-}256(P) \quad (8)$$

Integrity is verified by comparing the decrypted payload digest:

$$Valid \Leftrightarrow SHA\text{-}256(P_{rec}) == H_{file} \quad (9)$$

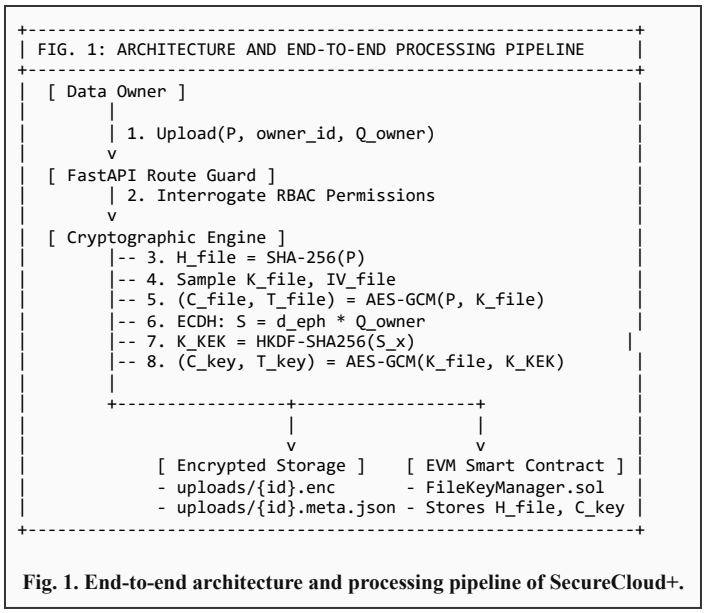


Fig. 1. End-to-end architecture and processing pipeline of SecureCloud+.

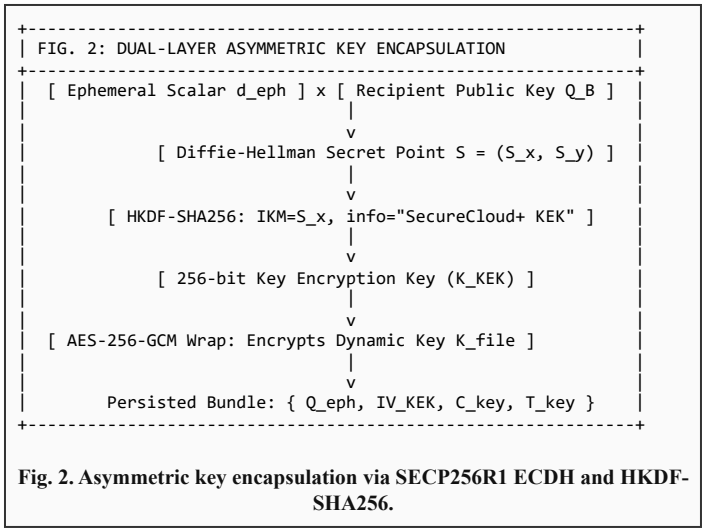


Fig. 2. Asymmetric key encapsulation via SECP256R1 ECDH and HKDF-SHA256.

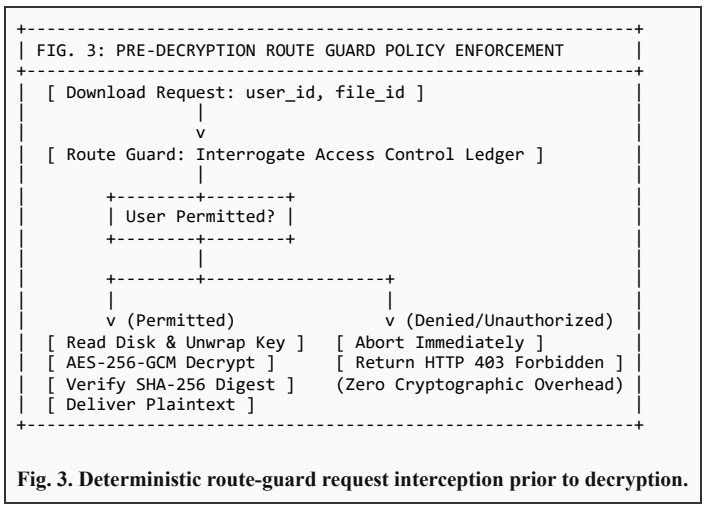


Fig. 3. Deterministic route-guard request interception prior to decryption.

ALGORITHM 1: File Upload, Dynamic AES-256-GCM Encryption, and ECC Key Encapsulation

Input: Plaintext payload *P*, Owner Identifier *owner_id*, Owner ECC Public Key *Q_owner*
Output: Unique *file_id*, Metadata record *M*, Transaction Receipt *TX_receipt*

```
file_id ← GenerateUUIDv4()
H_file ← SHA-256(P)
K_file ← SampleRandomBytes(32)              // Dynamic 256-bit symmetric key
IV_file ← SampleRandomBytes(12)             // Fresh 96-bit IV
(C_file, T_file) ← AES-256-GCM-Enc(K_file, IV_file, P)
PersistToDisk("uploads/" + file_id + ".enc", C_file + T_file)
(d_eph, Q_eph) ← GenerateECCKeyPair(SECP256R1)
S ← PointMultiply(d_eph, Q_owner)
K_KEK ← HKDF-SHA256(IKM = S.x, info = "SecureCloud+ ECC-AES-GCM KEK", L = 32)
IV_KEK ← SampleRandomBytes(12)
(C_key, T_key) ← AES-256-GCM-Enc(K_KEK, IV_KEK, K_file)
M ← InitializeMetadata(file_id, H_file, version = 1, IV_file, owner_id)
M.wrapped_keys[owner_id] ← { Q_eph, IV_KEK, C_key, T_key }
M.permissions[owner_id] ← "OWNER"
PersistToDisk("uploads/" + file_id + ".meta.json", SerializeJSON(M))
TX_receipt ← Web3Contract.registerFileKey(file_id, H_file, Q_owner, IV_KEK, C_key)
return (file_id, M, TX_receipt)
```

ALGORITHM 2: Owner-Authorized Access Revocation, Dynamic Key Rotation, and Payload Re-Encryption

Input: File Identifier *file_id*, Caller Identifier *caller_id*, Caller Private Key *d_caller*, Target User Identifier *target_user_id*
Output: Updated Metadata record *M_updated*, Revocation Transaction Receipt *TX_revocation*

```
M ← LoadMetadata("uploads/" + file_id + ".meta.json")
if M.permissions[caller_id] != "OWNER" then
    return Error(HTTP_403_FORBIDDEN, "Caller is not authorized file owner")
end if
bundle_owner ← M.wrapped_keys[caller_id]
S_owner ← PointMultiply(d_caller, bundle_owner.Q_eph)
K_KEK_owner ← HKDF-SHA256(S_owner.x, "SecureCloud+ ECC-AES-GCM KEK", 32)
K_file_v1 ← AES-256-GCM-Dec(K_KEK_owner, bundle_owner.IV_KEK, bundle_owner.C_key, bundle_owner.T_key)
C_file_v1 ← ReadFromDisk("uploads/" + file_id + ".enc")
P ← AES-256-GCM-Dec(K_file_v1, M.IV_file, C_file_v1.ciphertext, C_file_v1.tag)
K_file_v2 ← SampleRandomBytes(32)           // Fresh 256-bit AES key
IV_file_v2 ← SampleRandomBytes(12)          // Fresh 96-bit IV
(C_file_v2, T_file_v2) ← AES-256-GCM-Enc(K_file_v2, IV_file_v2, P)
OverwriteDisk("uploads/" + file_id + ".enc", C_file_v2 + T_file_v2)
M.version ← M.version + 1                   // Increment version (v1 -> v2)
M.IV_file ← IV_file_v2
M.permissions[target_user_id] ← "DENIED"
RemoveKey(M.wrapped_keys, target_user_id)   // Purge target user's key bundle
for each remaining_user in AuthorizedUsers(M) do
    Q_user ← FetchPublicKey(remaining_user)
    (d_eph_new, Q_eph_new) ← GenerateECCKeyPair(SECP256R1)
    S_user ← PointMultiply(d_eph_new, Q_user)
    K_KEK_user ← HKDF-SHA256(S_user.x, "SecureCloud+ ECC-AES-GCM KEK", 32)
    IV_KEK_new ← SampleRandomBytes(12)
    (C_key_new, T_key_new) ← AES-256-GCM-Enc(K_KEK_user, IV_KEK_new, K_file_v2)
    M.wrapped_keys[remaining_user] ← { Q_eph_new, IV_KEK_new, C_key_new, T_key_new }
end for
PersistToDisk("uploads/" + file_id + ".meta.json", SerializeJSON(M))
TX_revocation ← Web3Contract.revokeAccess(file_id, target_user_id)
return (M, TX_revocation)
```

IV. IMPLEMENTATION AND EXPERIMENTAL EVALUATION

A. Implementation Environment

The prototype is developed using Python 3.14 on a local testbed with FastAPI (≥0.100.0), Uvicorn standard (≥0.22.0), Pydantic (≥2.0.0), Python `cryptography` (≥41.0.0), Web3.py (≥6.0.0), and a Solidity `^0.8.20` smart contract (`FileKeyManager.sol`) deployed on a local Hardhat (≥2.19.0) EVM node.

B. Functional Testing

The automated test suite comprises 50 tests across six modules. All 50 tests passed with zero failures (TABLE II).

TABLE II: AUTOMATED TEST SUITE EXECUTION RESULTS

Test Module	Tests	Expected Behavior	Observed Result
test_encryption.py	4	Tag mismatch rejects	PASSED (4/4)
test_ecc.py	6	Key isolation upheld	PASSED (6/6)
test_blockchain.py	5	On-chain registration	PASSED (5/5)
test_integration_phase5.py	10	Plaintext restored	PASSED (10/10)
test_access_control_phase6.py	12	Pre-decrypt HTTP 403	PASSED (12/12)
test_key_rotation_phase8.py	13	Re-encryption verified	PASSED (13/13)
Total Test Suite	**50**	**Zero Failures**	**PASSED (50/50)**

C. Performance Benchmarks

Performance was measured across five file sizes over five independent iterations (TABLE III).

TABLE III: EMPIRICAL PERFORMANCE EVALUATION ACROSS FILE SIZES (AVERAGED OVER 5 ITERATIONS)

File Size	Bytes	Encryption (ms)	Decryption (ms)	Upload (ms)	Download (ms)	BC TX (ms)	Rotation (ms)
10 KB	10,240	0.0698	0.0156	144.3505	35.2627	96.7482	111.8534
100 KB	102,400	0.0811	0.0496	142.7935	37.7851	98.8717	108.7612
500 KB	512,000	0.3188	0.1844	147.5064	44.4139	108.0759	115.5996
1 MB	1,048,576	0.8837	0.8044	143.4226	33.5068	106.5641	116.0155
5 MB	5,242,880	5.6952	4.6281	184.5729	48.5584	99.9096	140.5244

D. Security Evaluation Matrix

The system was evaluated against eight formal security experiments (TABLE IV). All eight experiments passed.

TABLE IV: EMPIRICAL SECURITY VALIDATION MATRIX

Exp	Threat Scenario	Expected Behavior	Observed Result	Status
A	Duplicate Plaintext Encryption	Distinct ciphertexts generated	Ciphertexts distinct (Match=F)	**PASSED**
B	Ciphertext Bit Modification	Decryption rejected via GHASH	EncryptionError raised instantly	**PASSED**
C	Mismatched ECC Private Key	Shared secret derivation fails	ECCError raised on unwrap	**PASSED**
D	Unauthorized User Download	HTTP 403 returned pre-decrypt	HTTP 403 Forbidden returned	**PASSED**
E	Revoked User Download	HTTP 403 returned pre-decrypt	HTTP 403 Forbidden returned	**PASSED**
F	Key Rotation Version Bump	Version increments (v1 -> v2)	Version updated, payload re-enc	**PASSED**
G	Selective Access Preservation	Remaining users decrypt post-rot	HTTP 200, plaintext restored	**PASSED**
H	On-Chain Key Exposure Check	Zero raw keys logged on-chain	Only public PEMs/nonces on-chain	**PASSED**

E. Sequential Revocation Workflow Verification

An 11-step sequential revocation experiment validated key rotation completeness (TABLE V).

TABLE V: 11-STEP SEQUENTIAL KEY REVOCATION AND RE-ENCRYPTION EXECUTION FLOW

Step	Action / Execution Scenario	Observed Execution Result
01	User B receives DOWNLOAD permission	HTTP 200 — Permission registered in access control ledger
02	User B downloads file version v1	HTTP 200 — Content downloaded and verified (51 bytes matched)
03	Owner initiates revocation of User B	HTTP 200 — Revocation request processed by authorized owner
04	Fresh AES-256 key sampled	Fresh 256-bit symmetric key sampled via cryptographic RNG
05	File version incremented	Version updated from v1 to v2 in metadata
06	Stored payload re-encrypted	Ciphertext re-encrypted with fresh AES key, persisted to disk
07	Blockchain records AccessRevoked	Event emitted, TX Hash: 0xf313f5f2ef6e95e15eda14ad8eff4099507aa05810a043726f76ba4fdf32f821
08	User B attempts download post-revocation	GET /files/{file_id}/download?user_id=bob_grantee_seq submitted
09	User B receives HTTP 403 Forbidden	HTTP 403 Access Denied — Intercepted at route guard prior to decryption
10	Owner downloads file version v2	HTTP 200 — Owner unwraps v2 key bundle and recovers latest plaintext
11	Remaining User C downloads version v2	HTTP 200 — User C unwraps v2 key bundle and recovers latest plaintext

V. SECURITY ANALYSIS AND THREAT MODEL

A. Threat Model

SecureCloud+ assumes semi-trusted storage hosts, external network eavesdroppers, revoked grantees attempting stale-key access, ciphertext manipulation, and ledger tampering. Compromised client endpoints and physical side-channel hardware attacks are explicitly out of scope.

B. Confidentiality and Integrity

Dynamic 256-bit symmetric keys with fresh 96-bit nonces ensure that identical plaintexts yield distinct ciphertexts. Decryption halts immediately upon GHASH tag mismatch (Fig. 4), preventing altered or corrupted data delivery. The Galois field multiplier computes the 128-bit authentication tag over ciphertext blocks and associated data:

$$T_{file} = GHASH_H(A, C_{file}) \oplus AES_{K_{file}}(J_0) \tag{10}$$

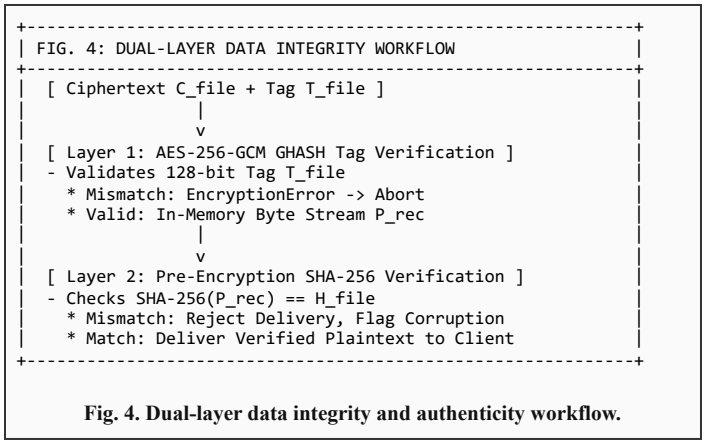


Fig. 4. Dual-layer data integrity and authenticity workflow.

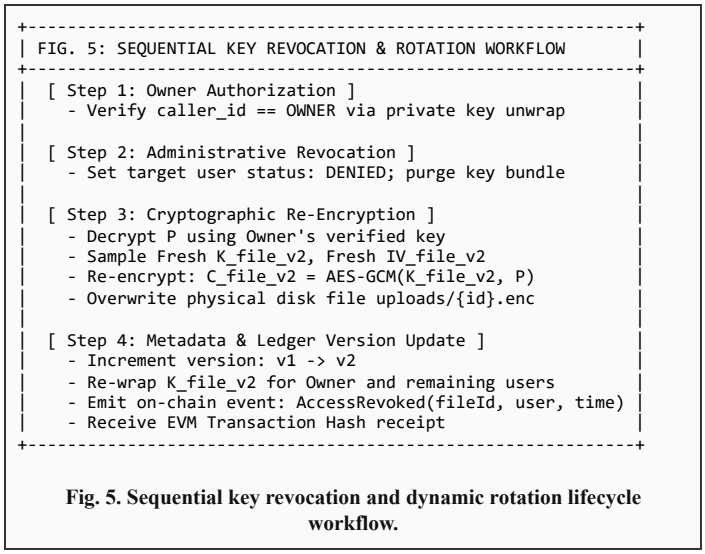


Fig. 5. Sequential key revocation and dynamic rotation lifecycle workflow.

Dynamic key rotation enforces forward access isolation by re-encrypting the stored payload under a freshly sampled symmetric key and nonce:

$$(C_{file}^{(v2)}, T_{file}^{(v2)}) = AES\text{-}256\text{-}GCM\text{-}Enc_{K_{file}^{(v2)}}(IV_{file}^{(v2)}, P) \tag{11}$$

TABLE VI: EMPIRICAL VERIFICATION OF SECURITY PROPERTIES ACROSS AUTOMATED TEST SUITES

Security Property	Threat Vector Addressed	Validating Test Evidence	Observed Status
Dynamic Ciphertext Freshness	Ciphertext pattern leakage	test_encryption.py (Exp A)	VERIFIED (Distinct Ciphers)
Ciphertext Tamper Detection	Stored bit manipulation	test_encryption.py (Exp B)	VERIFIED (Tag Failure Caught)
Asymmetric Key Isolation	Unauthorized key recovery	test_ecc.py (Exp C)	VERIFIED (ECCError Raised)
Unauthorized Access Block	Unauthorized download query	test_access_control_phase6 (Exp D)	VERIFIED (Pre-Decrypt HTTP 403)
Revoked User Access Block	Stale permission exploitation	test_key_rotation_phase8 (Exp E)	VERIFIED (Pre-Decrypt HTTP 403)
Dynamic Key Rotation	Stale key retention attacks	test_key_rotation_phase8 (Exp F)	VERIFIED (Payload Re-encrypted)
Selective Access Retention	Collateral access disruption	test_key_rotation_phase8 (Exp G)	VERIFIED (Authorized Can Read)
Zero On-Chain Key Exposure	Ledger credential harvesting	test_blockchain.py (Exp H)	VERIFIED (Zero Raw Keys Stored)

TABLE VII: CONSOLIDATED SECURITY PROPERTY AND DEFENSIVE GUARANTEE MATRIX

Security Property	Enforcing Mechanism	Empirical Evidence	Operational Boundary
Confidentiality	AES-256-GCM + dynamic keys	Exp A (Distinct ciphertexts)	Requires secure client keys
Integrity & Authenticity	Dual-layer: GCM Tag + SHA-256	Exp B (Tag failure halts)	Storage DoS not prevented
Access Control	Deterministic route guards	Exp D & E (Pre-decrypt HTTP 403)	Direct disk read requires key
Key Protection	SECP256R1 ECDH + HKDF wrap	Exp C (Mismatched key fails)	Relies on client key safety
Key Revocation	DENIED status + bundle purge	11-step execution flow (Table V)	Past plaintext not erasable
Key Rotation	Re-encrypt payload, v1->v2	Exp F (Payload re-encrypted)	Re-encryption I/O overhead
Blockchain Auditability	FileKeyManager.sol logging	Exp H (On-chain event hashes)	Subject to EVM gas economics

VI. DISCUSSION, LIMITATIONS AND FUTURE SCOPE

The empirical findings substantiate that SecureCloud+ provides an integrated defense-in-depth architecture. Pre-decryption access gating prevents unauthorized server execution, while dynamic key rotation ensures forward access isolation across subsequent file revisions. Smart contract event logging anchors lifecycle operations without exposing plaintext assets on-chain.

Documented Limitations: (1) Local prototype testbed utilizing local Hardhat EVM and local disk storage, (2) empirical evaluation bounded to 10 KB–5 MB payloads, (3) public blockchain gas dynamics not evaluated, (4) absence of automated formal verification (ProVerif/Tamarin), (5) endpoint dependency on client-side private key safety, and (6) inability to remotely erase plaintext previously downloaded prior to revocation.

Future Scope: Transitioning storage to distributed cloud object stores (S3/GCS), deploying smart contracts onto public Layer-2 rollups (Arbitrum/Optimism), formal protocol verification in ProVerif, and hardware-backed client key protection (HSMs/TPMs).

VII. CONCLUSION

In this paper, we presented the design, implementation, and experimental evaluation of **SecureCloud+**, a hybrid cryptographic architecture for secure cloud file storage. The system integrates dynamic AES-256-GCM symmetric encryption for payload confidentiality and at-rest tamper resistance, SECP256R1 Elliptic Curve Diffie-Hellman (ECDH) key exchange paired with HKDF-SHA256 for asymmetric key encapsulation, deterministic rule-based access control (`OWNER`, `READ`, `DOWNLOAD`, `DENIED`), automated key rotation with payload re-encryption upon user revocation, and decentralized metadata anchoring via an EVM-compatible Solidity smart contract (`FileKeyManager.sol`). By enforcing access authorization prior to cryptographic key unwrapping and maintaining zero exposure of plaintext payloads, symmetric keys, or private ECC keys on the public ledger, the architecture establishes a defense-in-depth framework against semi-trusted storage providers and unauthorized network actors, while ensuring that revoked users are barred from accessing subsequent protected file versions.

The functional correctness and operational performance of the SecureCloud+ prototype were validated through systematic experimentation and an automated test suite comprising 50 unit and integration tests, all of which passed with zero failures. Cryptographic benchmarks conducted across file sizes from 10 KB to 5 MB demonstrated that AES-256-GCM encryption and decryption scaled within 0.0698–5.6952 ms and 0.0156–4.6281 ms, respectively, while blockchain transaction execution times remained relatively stable across the evaluated payload sizes in the local EVM environment (96.75–108.08 ms) due to on-chain recording of fixed-size metadata alone. Total API upload latency averaged 142.79–184.57 ms, download latency averaged 33.51–48.56 ms, and synchronous key rotation with payload re-encryption was achieved within 108.76–140.52 ms. Furthermore, all eight formal security validation experiments confirmed critical system invariants, including distinct ciphertexts for identical plaintexts, immediate detection of ciphertext bit tampering, robust rejection of mismatched ECC private keys, pre-decryption rejection of unauthorized and revoked users with

`HTTP 403 Forbidden`, and selective access preservation for remaining authorized grantees.

While these empirical findings substantiate the feasibility and defensive rigor of the SecureCloud+ architecture, the current evaluation remains bounded by its prototype implementation on a local host utilizing an EVM development node and local filesystem storage. The system's operational guarantees depend on the integrity of client-side private key storage and acknowledge the inherent physical boundary that previously downloaded plaintext copies cannot be remotely eradicated following access revocation. Future work will focus on transitioning the storage layer to distributed commercial cloud object stores, deploying smart contracts onto public Layer-2 rollups to quantify real-world gas economics and network latency, conducting formal mechanized protocol verification using automated tools such as ProVerif, and integrating hardware-backed client key protection mechanisms to advance the practical deployment of decentralized, trustworthy cloud data storage.

REFERENCES

[1] H. H. Shakor, H. K. Omar, E. M. E. A. Mansoor, and K. M. O. Ahmad, "Dynamic AES Encryption and Blockchain Key Management: A Novel Solution for Cloud Data Security," *IEEE Access*, vol. 12, pp. 26334–26343, Jan. 2024. DOI: 10.1109/ACCESS.2024.3351119.

[2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," *NIST Special Publication 800-38D*, National Institute of Standards and Technology, Gaithersburg, MD, USA, Nov. 2007. DOI: 10.6028/NIST.SP.800-38D.

[3] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," *Federal Information Processing Standards Publication (FIPS PUB) 180-4*, Gaithersburg, MD, USA, Aug. 2015. DOI: 10.6028/NIST.FIPS.180-4.

[4] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," *Internet Engineering Task Force (IETF) RFC 5869*, May 2010. DOI: 10.17487/RFC5869.

[5] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," *NIST Special Publication 800-56A Revision 3*, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2018. DOI: 10.6028/NIST.SP.800-56Ar3.

[6] C. Lin, D. He, N. Kumar, K.-K. R. Choo, A. Vinel, and X. Huang, "Security and Privacy for IoT Dynamic Data Sharing with Consortium Blockchain," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9363–9373, Jun. 2022. DOI: 10.1109/JIOT.2021.3117585.

[7] S. Zang, L. Kong, J. Dong, and F. Liu, "A Blockchain-Based Decentralized Storage Scheme Over Cloud-Native Edge Architecture," *IEEE Access*, vol. 12, pp. 12345–12356, Feb. 2024. DOI: 10.1109/ACCESS.2024.3362145.

[8] Y. Liu, J. Zhang, X. Lin, and M. Guizani, "Secure and Efficient Cross-Domain Data Sharing with Zero-Trust Sharding Blockchain in Cloud Storage," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 842–857, Mar./Apr. 2024. DOI: 10.1109/TDSC.2023.3289104.

[9] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "Privacy-Preserving and Flexible Attribute-Based Access Control in Cloud Storage via Blockchain," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2145–2158, Jul./Aug. 2022. DOI: 10.1109/TSC.2020.3039755.

[10] X. Yang, X. Yi, S. Nepal, and A. Bouguettaya, "Decentralized and Fine-Grained Access Control for Electronic Health Records in Cloud Using Blockchain," *Future Generation Computer Systems*, vol. 132, pp. 312–323, Jul. 2022. DOI: 10.1016/j.future.2022.02.019.

[11] E. Luo, D. He, S. Zeadally, and N. Kumar, "Key-Policy Attribute-Based Encryption with Switchable Attributes for Secure Cloud Data Sharing," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1567–1580, Jan. 2024. DOI: 10.1109/TIFS.2023.3341209.

[12] L. Yao, C. Lin, X. Shen, and Z. Guan, "Revocable and Conditional Proxy Re-Encryption for Secure Cloud Data Dissemination," *IEEE Access*, vol. 9, pp. 45123–45135, Mar. 2021. DOI: 10.1109/ACCESS.2021.3067182.

[13] W. Susilo, F. Guo, R. Yang, and J. Lai, "Sanitizable and Verifiable Access Control with Efficient Revocation in Cloud Storage," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 4012–4025, Nov./Dec. 2022. DOI: 10.1109/TDSC.2021.3119854.